

UNIT II: Fundamentals of Neural Network

Q1. Describe the structure and working of an artificial neuron with a diagram.

An artificial neuron is a fundamental unit of a neural network inspired by the biological neuron in the human brain. It mimics the process of receiving inputs, processing them, and generating an output signal based on certain conditions.

Structure:

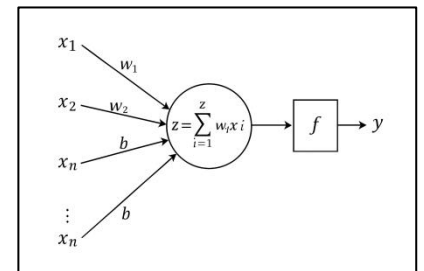
- **Inputs ($x_1, x_2, \dots, x_n$):** These are numerical signals or feature values fed into the neuron.
- **Weights ($w_1, w_2, \dots, w_n$):** Each input is multiplied by a corresponding weight representing its importance.
- **Bias (b):** A constant added to the weighted sum to allow shifting of the activation function.
- **Summation function:** Computes the weighted sum of inputs plus bias:
$$z = \sum_{i=1}^n w_i x_i + b$$
- **Activation function (f):** Applies non-linear transformation to the sum, producing neuron's output: $y = f(z)$

Working:

1. Each input is multiplied by its weight.
2. All weighted inputs are summed along with the bias.
3. The sum is passed through an activation function to introduce non-linearity.
4. The output y is forwarded to the next layer or used as the final result.

Diagram explanation:

- Multiple inputs $x_1, x_2, \dots, x_n$ enter the neuron.
- Each input is multiplied by a weight $w_1, w_2, \dots, w_n$.
- The weighted inputs are summed with bias b inside the summation node.
- The sum is passed to an activation function f .
- The activated output y is generated.



Q2. Explain different learning rules used in neural networks.

Learning in neural networks involves adjusting weights and biases to minimize the error between the predicted output and the actual output. Learning rules define how these adjustments are made based on the input data and error signals. Different learning rules are applied depending on the type of neural network and learning approach.

Types of Learning Rules:

1. **Hebbian Learning Rule:** Based on the principle proposed by Donald Hebb (1949), often summarized as "neurons that fire together, wire together." It is an unsupervised learning rule where the connection strength between two neurons is increased if they are activated simultaneously.
 - The weight update rule can be expressed as:
$$\Delta w_{ij} = \eta \cdot x_i \cdot y_j$$
 - where η is the learning rate, x_i is the input, and y_j is the output of the neuron.
 - This rule is used mainly in associative memory networks and self-organizing maps.
2. **Perceptron Learning Rule:** A supervised learning rule used in single-layer perceptrons. Weights are updated based on the difference between the actual output and the desired output.
 - The update formula is:
$$w_i^{new} = w_i^{old} + \eta(d - y)x_i$$
 - where d is the desired output, y is the actual output, and η is the learning rate.
 - The perceptron learning rule guarantees convergence if the data is linearly separable.

3. **Delta Rule (Widrow-Hoff Rule):** An extension of the perceptron learning rule for networks with continuous activation functions like sigmoid. It uses gradient descent to minimize the mean squared error between actual and desired outputs.
 - The weight update is: $\Delta w_i = \eta(d - y)x_i$
 - This rule forms the basis for more advanced algorithms like backpropagation.
4. **Backpropagation Algorithm:** This is the most widely used learning algorithm for multi-layer neural networks. It's a form of gradient descent that works in two phases:
 - **Forward Pass:** The network processes input data and produces an output.
 - **Backward Pass:** The error (or loss) is calculated at the output layer. The algorithm then propagates this error backward through the network using the chain rule of calculus to calculate the gradient of the loss function with respect to each weight. These gradients indicate the direction and magnitude of the change needed for each weight to reduce the error. The weights and biases are then updated using these gradients. This process is repeated for many iterations until the network's performance is satisfactory
5. **Competitive Learning Rule:** An unsupervised learning rule where neurons compete to become active. Only the neuron with the highest activation (winner) gets to update its weights to match the input pattern. Used in networks like Kohonen's Self-Organizing Maps (SOM).

Q3. What is an activation function? Give examples. Explain its use in neural networks.

An **activation function** is a mathematical function that introduces non-linearity into a neural network. It's a critical component of an artificial neuron that determines the output of the neuron based on the weighted sum of its inputs and a bias

An **activation function** ϕ transforms the input $\text{net} = \sum w_i x_i + b$ into an output $y = \phi(\text{net})$. Without activation functions, the neural network would behave like a linear regression model regardless of the number of layers.

Types and Examples of Activation Functions:

1. **Step Function (Threshold Function):** $\phi(\text{net}) = \begin{cases} 1 & \text{if } \text{net} \geq \theta \\ 0 & \text{if } \text{net} < \theta \end{cases}$ where θ is the threshold. Used in early perceptrons; produces binary output.
2. **Linear Activation Function:** $\phi(\text{net}) = \text{net}$ Output is proportional to input. Limited use because it does not introduce non-linearity.
3. **Sigmoid Function:** $\phi(\text{net}) = \frac{1}{1 + e^{-\text{net}}}$ Outputs values between 0 and 1. Smooth and differentiable, suitable for binary classification.
4. **Hyperbolic Tangent (tanh):** $\phi(\text{net}) = \tanh(\text{net}) = \frac{e^{\text{net}} - e^{-\text{net}}}{e^{\text{net}} + e^{-\text{net}}}$ Outputs values between -1 and 1. Zero-centered, which often helps training.
5. **Rectified Linear Unit (ReLU):** $\phi(\text{net}) = \max(0, \text{net})$ Allows faster and more effective training by introducing sparsity and avoiding vanishing gradients.

Use of Activation Functions in Neural Networks:

- **Non-linearity:** Activation functions enable neural networks to model non-linear relationships, which is essential for tasks like image recognition, natural language processing, etc.
- **Decision Making:** They help decide whether a neuron should be activated or not, based on the input signals.
- **Bounded Output:** Functions like sigmoid or tanh squash the output to a bounded range, making it easier to interpret as probabilities.
- **Gradient-Based Learning:** Differentiable activation functions (sigmoid, tanh, ReLU) enable the use of gradient descent and backpropagation for training deep networks.

Activation functions play a critical role by introducing non-linearity into the neural network, allowing it to learn complex mappings between inputs and outputs. Different activation functions serve various purposes, and the choice depends on the network architecture and task.

Q4. What is a perceptron? Describe perceptron learning algorithm.

A **perceptron** is the simplest form of an artificial neuron and a type of linear classifier. It was developed in the 1950s and is capable of solving problems that are **linearly separable**. It takes multiple inputs, multiplies them by weights, sums them up, and passes the result through a step-function activation function to produce a binary output (typically 0 or 1).

What is a Perceptron?

- It consists of one layer of artificial neurons (often just one neuron).
- Inputs are multiplied by weights, summed, and passed through a step activation function (threshold function) to produce a binary output (0 or 1).
- It can classify linearly separable data by finding a hyperplane separating the classes.

Mathematical Model:
$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right)$$

where ϕ is the step function:
$$\phi(\text{net}) = \begin{cases} 1 & \text{if } \text{net} \geq 0 \\ 0 & \text{if } \text{net} < 0 \end{cases}$$

Perceptron Learning Algorithm: The perceptron learning algorithm iteratively adjusts weights to minimize classification errors on training data.

Steps:

1. **Initialize weights and bias:** Set weights w_i and bias b to small random values or zeros.
2. **Input training sample:** Present an input vector $x = (x_1, x_2, \dots, x_n)$ with desired output $d \in \{0,1\}$.
3. **Compute output:** Calculate weighted sum $\text{net} = \sum w_i x_i + b$, and apply the step activation function to get predicted output y .
4. **Update weights:** If output y differs from desired output d , update weights and bias using:

where η is the learning rate (a small positive constant).

$$\begin{aligned} w_i &\leftarrow w_i + \eta(d - y)x_i \\ b &\leftarrow b + \eta(d - y) \end{aligned}$$

5. **Repeat for all training samples:** Iterate over the entire training set multiple times until convergence or for a fixed number of epochs.

Working Principle:

- If the perceptron predicts correctly, weights remain unchanged.
- If it misclassifies, weights are adjusted to reduce error, nudging the decision boundary in the right direction.
- The algorithm converges if the data is linearly separable.

Limitations: Can only solve linearly separable problems like AND/OR. Cannot solve non-linear problems like XOR.

The perceptron is a simple neural model for binary classification. The perceptron learning algorithm iteratively updates weights to correctly classify input patterns by minimizing error using a step activation function.

Q5. Compare linear and non-linear activation functions with suitable examples.

Activation functions determine the output of a neuron given its input. They can be broadly classified into **linear** and **non-linear** types, each with distinct properties and uses.

1. Linear Activation Function: A linear activation function outputs a value proportional to its input. Mathematically: $\phi(\text{net}) = k \cdot \text{net}$ where k is a constant, often set to 1.

- **Characteristics:** Output is unbounded and directly proportional to input. Simple, computationally efficient.
- **Limitations:** No matter how many layers the network has, stacking linear activation functions results in a model that behaves like a single-layer linear model.
 - Cannot model complex non-linear relationships.
 - Makes the network equivalent to a linear regression.
- **Use Case:** Primarily used in the output layer for regression tasks where continuous outputs are needed.

2. Non-linear Activation Function: Non-linear activation functions produce outputs that are non-linear transformations of their inputs. This enables the neural network to learn complex, non-linear mappings.

- **Sigmoid:** $\phi(net) = \frac{1}{1 + e^{-net}}$ Output range: (0, 1)
- **Hyperbolic Tangent (tanh):** $\phi(net) = \tanh(net) = \frac{e^{net} - e^{-net}}{e^{net} + e^{-net}}$ Output range: (-1, 1)
- **Rectified Linear Unit (ReLU):** $\phi(net) = \max(0, net)$ Output range: [0, ∞)
- **Characteristics:** Introduces non-linearity to the model.
 - Allows the network to learn and represent complex functions.
 - Differentiable (except at some points like ReLU at zero), enabling gradient-based optimization.
 - Helps in solving real-world problems with complex decision boundaries.
- **Use Case:** Used in hidden layers of neural networks for classification, regression, and deep learning tasks.

Aspect	Linear Activation	Non-linear Activation
Output	Proportional to input	Non-linear transformation of input
Complexity	Simple	Complex
Network Capability	Only linear problems	Can model complex, non-linear problems
Stacking Effect	Equivalent to single layer	Enables deep networks to learn features
Differentiability	Yes	Yes (mostly)
Examples	$y = x$	Sigmoid, tanh, ReLU
Usage	Output layer for regression	Hidden layers and classification output

Linear activation functions are simple but limited to modeling linear relationships, whereas non-linear activation functions enable neural networks to learn complex patterns and solve real-world problems effectively. Hence, non-linear activations are preferred in most hidden layers.

Q6. Differentiate between single-layer and multi-layer feed-forward networks.

Feed-forward neural networks are the simplest type of artificial neural networks where information flows only in one direction—from input to output—without loops. Depending on the number of layers, they are classified as single-layer or multi-layer networks.

Single-Layer Feed-Forward Network:

- **Structure:** Consists of only one layer of neurons (output layer) directly connected to the input layer. There are no hidden layers.
- **Operation:** The inputs are directly connected to the output neurons, which compute the weighted sum and apply an activation function (usually a step or linear function).
- **Learning Capability:** Can only solve **linearly separable problems** because it implements a linear decision boundary.
- **Examples:** Perceptron, Single-layer Adaline (Adaptive Linear Neuron)
- **Limitations:** Cannot solve complex problems like XOR which are not linearly separable.

Multi-Layer Feed-Forward Network (MLP - Multi-Layer Perceptron):

- **Structure:** Consists of an input layer, one or more hidden layers, and an output layer. Each layer consists of multiple neurons.
- **Operation:** Information flows from input to output passing through hidden layers, where each neuron performs a weighted sum followed by a non-linear activation function.

- **Learning Capability:** Can solve **both linear and non-linear problems** by learning complex mappings between inputs and outputs.
- **Examples:** Multi-layer perceptron trained using backpropagation algorithm.
- **Advantages:** Can approximate any continuous function (Universal Approximation Theorem). Suitable for tasks like image recognition, speech processing, and other complex classifications.
- **Disadvantages:** More computationally intensive due to multiple layers and parameters.

Feature	Single-Layer Feed-Forward Network	Multi-Layer Feed-Forward Network
Number of Layers	One (Output layer only)	Multiple (One or more hidden layers + output)
Complexity	Simple	Complex
Learning Capability	Can solve only linearly separable problems	Can solve both linear and non-linear problems
Activation Function	Usually linear or step function	Non-linear functions like sigmoid, ReLU
Training Algorithm	Perceptron learning rule	Backpropagation algorithm
Example Applications	Basic classification	Image processing, NLP, function approximation
Computational Cost	Low	High

Q7. Explain the working of a Recurrent Neural Network (RNN) with its applications.

Recurrent Neural Networks (RNNs) are a class of artificial neural networks designed to handle sequential or time-series data. Unlike feed-forward networks, RNNs have loops that allow information to persist, making them suitable for tasks where context and order matter.

Structure of RNN: Consists of input layer, one or more recurrent hidden layers, and output layer. Each neuron in the hidden layer receives input from the current time step and from its own output at the previous time step (feedback loop).

Working Principle:

- At time step t , the RNN takes input vector x_t and the hidden state from the previous time step h_{t-1} .
- The hidden state is updated as: $h_t = \phi(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$ where W_{xh} and W_{hh} are weight matrices, b_h is bias, and ϕ is the activation function (typically tanh or ReLU).
- The output at time t is computed as: $y_t = \psi(W_{hy}h_t + b_y)$ where W_{hy} is output weight matrix and ψ is usually softmax or linear function.
- This recurrence allows the network to maintain a **memory** of previous inputs via the hidden state h_t , enabling it to process sequences.

Key Characteristics:

- **Temporal Dynamics:** RNNs capture temporal dependencies in sequential data.
- **Parameter Sharing:** The same weights are used at every time step, reducing the number of parameters.
- **Backpropagation Through Time (BPTT):** A variant of backpropagation is used to train RNNs by unfolding the network across time steps.

Applications of RNNs:

1. **Natural Language Processing (NLP):** Language modeling, text generation, machine translation, and sentiment analysis.
2. **Speech Recognition:** Understanding spoken language by modeling audio sequences.
3. **Time Series Prediction:** Stock price forecasting, weather prediction, and sensor data analysis.
4. **Music Generation:** Composing music by learning patterns in note sequences.
5. **Video Analysis:** Activity recognition and video captioning by analyzing frames over time.

Limitations:

- **Vanishing and Exploding Gradients:** Difficulties in learning long-term dependencies.
- Addressed by advanced RNN variants like LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit).

Recurrent Neural Networks are powerful models for sequential data that use feedback connections to remember past information. Their ability to model time-dependent patterns makes them invaluable in many real-world applications involving sequences.

Q8. Discuss various types of neural network architectures in detail.

Neural networks come in various architectures designed to process different types of data such as static, sequential, or spatial data. Each architecture has unique features and is suited to specific problem types.

1. **Feed-Forward Neural Networks (FFNN):** Information flows in one direction from input to output without cycles. Comprises input layer, one or more hidden layers, and an output layer. Suitable for static data classification and regression problems. Examples include single-layer perceptron and multi-layer perceptron (MLP).
2. **Convolutional Neural Networks (CNN):** Specialized for processing grid-like data such as images. Uses convolutional layers to automatically learn spatial features like edges, textures, and objects. Includes convolutional, pooling, and fully connected layers. Widely used in image recognition, video analysis, and computer vision tasks.
3. **Recurrent Neural Networks (RNN):** Designed for sequential data processing by maintaining a hidden state that captures information from previous time steps. Effective for tasks where temporal context is important. Traditional RNNs suffer from vanishing and exploding gradients which limit long-term memory.
4. **Long Short-Term Memory (LSTM) Networks:** An advanced variant of RNN designed to overcome limitations of standard RNNs. Contains specialized memory cells and gates (forget, input, output) that control information flow. Capable of learning long-term dependencies in sequences. Used in language modeling, speech recognition, time series forecasting.
5. **Gated Recurrent Unit (GRU) Networks:** Similar to LSTM but with a simpler structure using only two gates: reset and update gates. Requires fewer parameters and is faster to train than LSTM while still managing long-term dependencies well. Applied in similar domains as LSTM.
6. **Radial Basis Function Networks (RBFN):** Employ radial basis functions as activation in hidden layers. Typically three-layered (input, hidden with RBF neurons, output). Used for function approximation, regression, and classification.
7. **Modular Neural Networks (MNN):** Composed of multiple independent modules or subnetworks, each solving part of a problem. Modules' outputs are combined for final decision-making. Useful for complex problems decomposed into simpler tasks.
8. **Self-Organizing Maps (SOM):** Unsupervised networks that map high-dimensional input data into low-dimensional (usually 2D) maps preserving topological properties. Used for clustering, visualization, and dimensionality reduction.
9. **Autoencoders:** Unsupervised neural networks designed to learn compressed representations of data. Consist of encoder and decoder parts. Useful for dimensionality reduction, anomaly detection, and denoising.

Neural network architectures vary widely, from simple feed-forward networks for static data to complex recurrent networks like LSTM and GRU for sequential data. CNNs excel in spatial data analysis, while unsupervised architectures like SOM and autoencoders serve clustering and compression roles. Selecting the right architecture depends on the data type and task requirements.

Q9. Describe how neural networks are used in music genre classification.

Music genre classification is the task of categorizing music tracks into predefined genres (e.g., rock, jazz, classical) based on audio characteristics. Neural networks are widely used for this purpose due to their ability to learn complex patterns from raw or processed audio data.

Process of Music Genre Classification Using Neural Networks:

1. **Feature Extraction:** Raw audio signals are transformed into feature representations such as Mel-Frequency Cepstral Coefficients (MFCCs), chroma features, spectral contrast, and rhythm patterns. These features capture timbre, pitch, rhythm, and other characteristics essential for genre discrimination.

2. **Input to Neural Network:** Extracted features are used as inputs to the neural network. Depending on architecture, inputs can be raw audio frames (in CNNs) or sequences of feature vectors (in RNNs).
3. **Neural Network Architecture:**
 - **Feed-Forward Networks (MLP):** Can be used with handcrafted features for classification.
 - **Convolutional Neural Networks (CNN):** Process spectrograms or other 2D representations to capture local and hierarchical audio patterns.
 - **Recurrent Neural Networks (RNN) / LSTM:** Model temporal dependencies and sequence information in music, capturing rhythm and progression.
4. **Training the Network:** The network is trained using labeled datasets where each music sample is tagged with a genre. Loss functions (e.g., cross-entropy) are minimized using gradient descent and backpropagation.
5. **Classification:** After training, the neural network predicts genre of unseen music samples on learned patterns.

Advantages of Neural Networks for Music Genre Classification:

- Ability to learn complex and subtle audio features automatically.
- Effective in modeling both spectral and temporal aspects of music.
- Superior performance compared to traditional machine learning methods relying on manual feature engineering.

Applications:

- Music streaming services for personalized recommendations.
- Automatic organization and tagging of large music libraries.
- Music information retrieval and analysis.

Neural networks are powerful tools for music genre classification. By extracting meaningful audio features and leveraging architectures like CNNs and RNNs, they learn to discriminate between genres accurately, enabling various practical applications in music technology.

Q10. Explain the role of activation functions in deep learning networks with diagrams.

Activation functions introduce non-linearity into neural networks, which is critical for deep learning models to learn complex mappings between inputs and outputs. Without activation functions, the network behaves like a linear model, regardless of its depth.

Role of Activation Functions in Deep Learning:

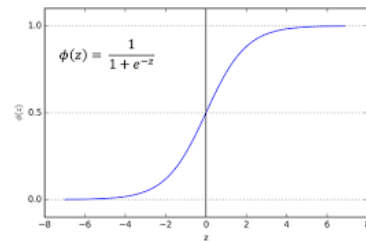
1. **Introducing Non-Linearity:** Real-world data often involve non-linear relationships. Activation functions enable neurons to learn these by applying non-linear transformations to inputs. This allows deep networks with multiple layers to approximate any continuous function (Universal Approximation Theorem).
2. **Controlling Signal Flow:** Activation functions determine whether a neuron should be activated based on its input, allowing selective passing of information.
3. **Facilitating Gradient-Based Optimization:** Activation functions must be differentiable so that algorithms like backpropagation can compute gradients and update weights effectively.
4. **Preventing Output Saturation:** Functions like ReLU reduce the problem of vanishing gradients common with sigmoid/tanh in deep networks, enabling faster and more stable training.

Common Activation Functions Used in Deep Learning:

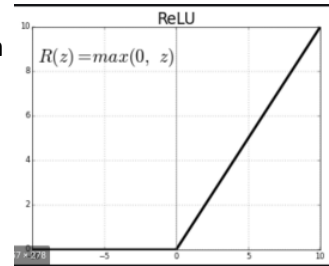
Activation Function	Formula	Output Range	Characteristics
Sigmoid	$\sigma(x) = 1 / (1 + e^{-x})$	(0, 1)	Smooth, saturates at extremes
Tanh	$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	(-1, 1)	Zero-centered, saturates at extremes

ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Sparse activation, efficient
Leaky ReLU	$f(x) = \max(\alpha x, x)$ (α small, e.g., 0.01)	$(-\infty, \infty)$	Allows small gradient when inactive

Sigmoid Activation Function



ReLU Activation Function



Q11. Write a short note on how neural networks learn patterns from data.

Neural networks learn by adjusting their internal parameters (weights and biases) to minimize the difference between predicted outputs and actual targets in the training data. This learning process enables the network to recognize underlying patterns and make accurate predictions on new, unseen data.

How Neural Networks Learn Patterns:

- 1. Initialization:** Weights and biases are initialized, usually with small random values, before training begins.
- 2. Forward Propagation:**
 - Input data is passed through the network layer by layer.
 - Each neuron calculates a weighted sum of its inputs and applies an activation function to produce output.
 - The final output layer produces the network's prediction.
- 3. Error Calculation:**
 - The predicted output is compared with the actual target using a loss function (e.g., mean squared error for regression, cross-entropy for classification).
 - The loss quantifies how well the network predicts the data.
- 4. Backward Propagation (Backpropagation):**
 - The error is propagated backward through the network.
 - Gradients of the loss function with respect to each weight are computed using calculus (chain rule).
 - This determines how much each weight contributes to the error.
- 5. Weight Update:**
 - Using optimization algorithms like gradient descent, weights are updated to reduce the error.
 - Updates are typically proportional to the negative gradient scaled by a learning rate.
- 6. Iteration:** Steps 2 to 5 are repeated over many epochs (complete passes through the training data) until the network converges or reaches a satisfactory accuracy.

Learning Outcome: Through this iterative process, the network tunes its weights to capture patterns and relationships in the input data, allowing it to generalize and make predictions on new inputs.